BSc (Hons) in Computing - 25.1

IOS APPLICATION DEVELOPMENT – COURCE WORK

Individual Report

Coventry Index: 16603596

NIBM Index: COBSCCOMP251P-045

NIBM Registered Name: G P M A N Jeewantha

Faculty of Engineering, Environment and Computing School of Computing, Electronics and Mathematics Coventry University School of Computing and Engineering National Institute of Business Management Colombo-7

Table of Contents

1. Overview of the app *2*

2. Development process *3*

Architecture **3**

How data moves **4**

Challenges and how they were handled **4**

Known limitations **6**

3. MVP implementation *7*

Mandatory platform feature 1 — User Notifications 7

Mandatory platform feature 2 — Face ID / Touch ID 7

Also implemented — Core Data 8

Design decisions **8**

4. Advanced iOS features *9*

Core ML — on-device solar forecast 9

WidgetKit — Home Screen snapshot 10

5. External API and how markers can use it *10*

Marker access 12

6. Above and beyond *13*

Multiple advanced features **13**

Architecture and resilience 13

Polish 14

Creativity 14

7. Unit tests, accessibility *14*

Unit tests 14

Accessibility 16

8. Learning outcomes reflection *18*

LO1 — Platform components across devices and formatss **18**

LO2 — Software suitable for a mobile architecture **19**

LO3 — External APIs and devices **19**

LO4 — Sensors and hardware **19**

9. Declaration of LLM use *20*

10. References *21*

1. Overview of the app

HelioLanka is a customer iOS app for people who already have rooftop solar connected to a metered installation in Sri Lanka. The problem it solves is simple: export readings, weather, and money sit in a backend, while the customer wants a phone they can open in the evening and understand the day without a spreadsheet.

The audience is a residential solar customer, not an installer and not a staff user. Staff accounts are recognised and stopped at a short “use the web admin” screen so the mobile app stays a customer product.

What the customer can do

Area	**What it does**
Sign in	Username or email, password, then a 6-digit one-time code. Optional Face ID, Touch ID, or Optic ID after that first successful sign-in.
Register	New customer account linked to a 10-digit account number.
Home	Selected meter, site name, province, district and substation, today’s export, live power, last interval, cumulative export, tariff, and the five newest intervals. Pull to refresh.
History	Daily export totals for 7, 30, or 90 days, with a bar chart, a total, and a daily average. Totals are stored on the device in Core Data.
Weather	Site forecast from the backend: today’s temperature, cloud, rain, an hourly temperature line, an hourly solar-radiation bar chart, and a 7-day outlook.
Earn	Earnings from the customer’s own history and the site tariff, plus a 30-day generation and income forecast produced on the device.
Calendar	Month grid of export days, notes, optional local reminders, high-generation highlighting, and a one-way sync of strong-sun days into the device calendar.
Account	Profile nickname, a map of the customer’s meters, a public leaderboard (day / week / month), biometric sign-in, and the Home Screen widget switch.
Widget	Small and medium Home Screen widgets showing today’s export and tomorrow’s forecast. The widget reads a snapshot from an App Group. It does not call the API and it does not hold a token.

The two Component 2 platform features I am submitting as the MVP baseline are User Notifications and biometric authentication (Face ID / Touch ID / Optic ID). Core Data is also implemented, for daily export totals and calendar notes. Core Data, notifications, and biometrics are not used as the Component 3 feature.

The Component 3 advanced features are Core ML (on-device solar forecast) and WidgetKit (a live Home Screen snapshot of that forecast and today's export). MapKit, EventKit, and the Charts framework are additional product features and are described under Above and Beyond.

2. Development process

The app was built in Xcode with Swift and SwiftUI, in phases that are still visible in the source comments: networking and environment, authentication, session state, customer screens, persistence, weather and earnings, the forecast model, calendar and notifications, then account, map, leaderboard, biometrics, and the widget.

Architecture

The structure is a small MVVM layout with actors for network work.

- *HelioLankaApp* owns one *SessionManager* and injects it with *.environment.*
- *ContentView* switches on *SessionManager.Phase*: launching, signed out, OTP, customer tabs, or staff blocked.
- *CustomerTabShell* is a six-tab *TabView*: Home, History, Weather, Earn, Calendar, Account.
- Each customer screen has a view and an *@Observable* view model (*HomeViewModel, HistoryViewModel, WeatherViewModel, EarnViewModel, CalendarViewModel, ProfileViewModel, LeaderboardViewModel*, plus the auth view models).
- Network calls live in actors: *APIClient, AuthService, InstallationService, ReadingService, WeatherService, ProfileService, LeaderboardService*. Using actors keeps token updates off the main actor and avoids overlapping refresh calls corrupting the token pair.
- Device services sit beside the network: *DailyGenerationStore, CalendarNoteStore, NotificationService, CalendarService, BiometricAuthService, BiometricCredentialStore, TokenStore, SolarForecastModel, WidgetPublisher*.

- *AppPreferences* stores the customer's switches (notifications, daily summary time, generation alerts, widget, biometrics) in *UserDefaults*. Secrets are not stored there.

I kept the view models on the main actor because they publish UI state. I kept the API client as an actor because a 401 can trigger a refresh and then a retry, and that sequence must not run twice at once on the same token.

How data moves

1. Login returns a challenge, not a session. Verify OTP returns access and refresh tokens.
2. Tokens are written to the Keychain (*TokenStore*, service *lk.heliolanka.tokens*).
3. Later requests send Authorization: Bearer. A 401 with a refresh token calls *POST /api/v1/auth/refresh* once and retries the original request.
4. Home loads the installation composite and the latest readings together with *async let*.
5. History, Earn, and Calendar pull several pages of readings (up to five pages of 100), sum *intervalExportKwh* by local calendar day, and upsert those days into Core Data. Rows older than 90 days are deleted.
6. Earn asks the weather endpoint for hourly conditions, rolls them into daily features, and runs the on-device forecaster for 30 days.
7. Home publishes today's export into the App Group snapshot. Earn publishes tomorrow's first forecast day into the same snapshot. The widget reloads its timeline.

Challenges and how they were handled

OTP before a session.

The backend does not return tokens on the password step. The app had to hold a challenge code, show a masked destination, and only then store tokens. A wrong code stays on the OTP screen. A cancelled biometric prompt falls back to the password form.

Token lifetime.

Access tokens expire. APIClient.rawRequest retries once after refresh. If refresh fails, the session is cleared on the next *me()* failure during launch, and the customer signs in again. Logout still attempts *POST /api/v1/auth/logout* and always clears the Keychain pair.

Date formats.

The API mixes ISO-8601 timestamps with and without fractional seconds. The decoder tries both before failing the payload.

Daily totals are not an API field.

Readings are interval rows. The phone sums them per local day inside *DailyGenerationStore.sync*. That sum is what History charts, what the calendar colours, and what the anomaly check compares. Doing it on the device means the chart still has the last 90 days after a later launch, without waiting for every page again before the first paint — the screen still refreshes from the network when it appears.

Forecast without a network model.

Component 3 does not allow a cloud LLM. The forecaster therefore runs in the app process. It prefers a compiled Core ML model (*SolarGenerationForecaster*), then a bundled gradient-boosting tree JSON (*json-gbdt*) with the same feature scaler, then a small radiation heuristic if neither file loads. The Earn screen prints the engine name (*coreml, json-gbdt*, or *heuristic*) so the running path is visible.

Widget must not become a second client.

A widget cannot safely store the Keychain session or prompt for Face ID. *WidgetPublisher* writes a *WidgetSnapshot* (meter, site name, today kWh, tomorrow kWh, timestamp) into the App Group *group.lk.heliolanka.app*. The widget timeline refreshes about every 30 minutes from that snapshot. If the customer has never opened Home and Earn, the widget asks them to open the app.

Biometric storage.

The password used for Face ID is held in memory only until the customer accepts or dismisses the offer (*pendingLoginPassword*). It is written to a separate Keychain service (*lk.heliolanka.biometric.credentials*) only after a successful biometric prompt. Turning the switch off deletes that item. It is never written to *UserDefaults*.

High and low generation alerts.

A single “today is low” rule is noisy on a new account. *GenerationAnomalyEvaluator* requires at least four stored days, three earlier positive days, and a baseline of at least 2 kWh. High means the latest day is at least 35% above that baseline and at least 3 kWh above it. Low means it is at most 55% of the baseline and at least 2.5 kWh below it. The same alert is not scheduled twice for the same day.

Calendar privacy.

Sync uses EventKit write-only access. The app adds all-day “strong sun” events. It does not read the customer’s other events. Notes stay in Core Data even if calendar permission is denied.

Staff versus customer.

UserMe.isCustomer checks the role. Any other role becomes *staffBlocked* and can only sign out.

Known limitations

- The Home tile labelled “Capacity” currently formats *tariffRate* as currency. The site model does carry *capacityKw*; the label and the value do not match. I treat this as a labelling bug, not as the capacity figure.
- Several titles use a fixed point size (for example 28 pt and 40 pt). Body text that uses *.headline, .subheadline*, and *.caption* follows Dynamic Type. The large titles do not.
- The app *calls .preferredColorScheme(.dark)*, so the adaptive light palette in *HelioTheme* is not what the customer sees. The widget uses its own light chip colours on purpose, because the navy token becomes light in Dark Mode and would disappear on a white chip.
- There is no *NavigationSplitView* or size-class branch. iPhone and iPad share one layout. Section 8 explains that choice.
- The unit-test target is not in the repository until I add it from the unit-test guide. I will not claim a green run until I have one.

3. MVP implementation

The MVP is the customer loop: sign in, see today's export, look back over recent days, see the weather at the site, estimate earnings, and keep notes. It follows the tab flow above.

Mandatory platform feature 1 — User Notifications

NotificationService uses *UNUserNotificationCenter*.

- Permission. Requested when a reminder, a generation alert, or the daily summary needs it. The usage string is in *Info.plist*: local reminders for notes, generation alerts, and the daily summary.
- Note reminders. Saving a calendar note with "Remind me on this day" schedules a one-shot calendar trigger at the chosen hour and minute (*note-<id>*). Deleting the note removes that request.
- Daily summary. A repeating calendar trigger, identifier *daily-summary*, default 18:00. The Calendar tab turns it on, picks the time, and can turn all notifications off. Launch reapplies the pending request from *AppPreferences*.
- Generation alerts. After the calendar loads history, a high or low day can post a banner about two seconds later, once per day, and only if both the master switch and the generation-alert switch are on. Banners are also presented while the app is in the foreground.

This is more than a single test notification. The triggers are tied to notes the customer writes and to export totals the app has stored.

Mandatory platform feature 2 — Face ID / Touch ID

BiometricAuthService uses *LAContext* and *deviceOwnerAuthenticationWithBiometrics*. The button title follows the device: Face ID, Touch ID, or Optic ID.

Flow:

1. Password login succeeds and the OTP is verified.
2. If biometrics exist and are not already enabled, the app offers to turn them on.
3. Accepting the offer prompts the system sheet, then stores the username and password in the Keychain.

4. Next launch, if the switch is on, the app asks for biometrics before calling */api/v1/auth/me* with the saved tokens.
5. The sign-in screen shows “Sign in with Face ID” when a credential or a token is available.
6. If the Keychain session is gone, biometrics still fill the password step, and the customer still completes the emailed OTP. Biometrics do not skip the server’s second factor.
7. Account → “Sign in with Face ID” can turn the feature off, which deletes the Keychain password.

Also implemented — Core Data

HelioPersistence builds the managed model in code (there is no *.xcdatamodeld* file) and stores it as SQLite under Application Support.

- *DailyExport*: key, meter id, day start, export kWh, updated at.
- *CalendarNoteRecord*: id, day, title, body, reminder flag, hour, minute, created and updated.

NSMergeByPropertyStoreTrumpMergePolicy is set so a re-sync of the same day updates the row rather than crashing on a conflict. This persistence is part of the product. It is not the feature I am offering for Component 3.

Design decisions

- Dark UI, teal brand, rounded cards, and the HelioLanka mark, so the six tabs feel like one product.
- One selected meter, changed from *MeterPicker* when the account has more than one. Home, History, Weather, and Earn all reload when *selectedMeterId* changes.
- Empty and error lines are on the screen (“No meter is linked”, “No export history yet”, “No forecast hours”, “No site coordinates yet”) instead of a silent blank view.
- Pull to refresh on the data tabs.
- Currency and energy use *HelioFormat* with the *en_LK* locale, so earnings read as LKR and energy as kWh.

4. Advanced iOS features

Push notifications, Core Data, and biometrics are excluded here, as the brief requires.

Core ML — on-device solar forecast

SolarForecastModel forecasts the next 7 days by default and 30 days on the Earn screen. Each day is a feature vector:

- installation *capacityKw*
- weekday (Monday = 0, matching the training convention), month, day of year
- lag 1 day, lag 7 days, 7-day mean, 7-day standard deviation of export
- weather: average temperature, cloud cover, humidity (the weather payload has no humidity, so the training default 75 is used), precipitation, shortwave radiation

InputScaler standardises those features with *input_scaler.json* (mean and scale per feature).

Core ML path.

CoreMLForecastEngine loads *SolarGenerationForecaster.mlmodelc* or compiles the *.mlmodel*, builds an *MLDictionaryFeatureProvider* from the scaled features, and reads *predicted_kwh*. Inference stays inside the app process. There is no call to an external chat or LLM API.

Fallback path.

If the compiled model is missing, *JSONTreeForecastEngine* walks the bundled tree JSON (*SolarGenerationForecaster.json*) with the same scaler, learning rate, and prior. If that file is missing too, a heuristic scales the 7-day mean by shortwave radiation. The fallback exists so a missing model file does not blank the Earn screen. The screen shows which engine ran.

The forecast is not a detached demo page. Earn uses it for the 30-day kWh total, the 30-day income (predicted kWh times the effective tariff), the bar chart, and the “tomorrow” number on the widget.

Weather features come from *GET /api/v1/installations/{meterId}/weather*. The model does not. The network supplies conditions. The prediction is on device. That split is deliberate: a remote model would not meet the on-device rule, and a forecast with no weather would ignore the reason a Sri Lankan rooftop day changes.

WidgetKit — Home Screen snapshot

HelioLankaWidget is a WidgetKit extension with small and medium families.

- *TimelineProvider* loads *WidgetSnapshot* from the App Group.
- Gallery previews use a fixed sample (Rooftop Site 1, 4.2 kWh today, 7.4 kWh tomorrow) and are not shown as live data.
- A snapshot with *updatedAt* == *.distantPast* shows "Open the app and visit Home and Earn".
- Small layout stacks the two metrics. Medium layout places them side by side.
- Account can turn the widget off or ask WidgetCenter to reload timelines.

The widget is comprehensive in the sense the brief asks for: the numbers are the customer's meter and the on-device forecast, updated when those screens load, and the empty state is explicit. It is not a hardcoded pin or a static label.

Why these two, together

The forecast answers "what might tomorrow produce?". The widget answers that question without unlocking the full app. Together they are the reason the app is more than a readings list.

5. External API and how markers can use it

The app talks to one HTTPS API. The base URL is set in *AppEnvironment.apiBaseURL*:

https://he-907259e87ef04b90924f651d03e9cf03.ecs.ap-southeast-1.on.aws

(https://he-907259e87ef04b90924f651d03e9cf03.ecs.ap-southeast-1.on.aws/swagger)

A debug build can override it with the environment variable *HELIO_API_BASE*. Release builds use the URL above.

The service is the HelioLanka customer API (AWS ECS, ap-southeast-1). It is not Firebase and it is not a public unauthenticated dataset. Weather, leaderboard, installations, and auth are all this API. The phone does not call Open-Meteo or any other weather host directly. The backend returns the hourly forecast already tied to the installation's latitude and longitude.

Endpoints the app uses

Method and path	Auth	Used for
POST /api/v1/auth/register	No	Create a customer
POST /api/v1/auth/login	No	Start login, returns an OTP challenge
POST /api/v1/auth/otp/verify	No	Exchange the code for access and refresh tokens
POST /api/v1/auth/otp/resend	No	Resend the code
POST /api/v1/auth/refresh	No (refresh token in the body)	New access token
GET /api/v1/auth/me	Bearer	Current user, role, linked meters
POST /api/v1/auth/logout	Bearer	End the session
POST /api/v1/auth/link-account	Bearer	Attach an account number
GET /api/v1/customers/me/profile	Bearer	Nickname
PATCH /api/v1/customers/me/profile	Bearer	Update nickname and public flag
GET /api/v1/installations/{meterId}/composite	Bearer	Site, tariff, capacity, coordinates, last reading
GET /api/v1/installations/{meterId}/readings	Bearer	Paged interval readings
GET /api/v1/installations/{meterId}/weather	Bearer	Hourly forecast for that site
GET /api/v1/leaderboard?period=&limit=50	No	Public ranks for daily, weekly, or monthly

Errors are decoded from message or description when the body is JSON. Transport and decode failures are shown as a short retry message rather than a raw stack trace.

Marker access

Markers need a customer role. A staff role only sees the blocked screen, which is useful as a negative test but not as a way to see the product.

Item	Value
Base URL	https://he-907259e87ef04b90924f651d03e9cf03.ecs.ap-southeast-1.on.aws
Username	customer
Password	Customer@12345
How to get the OTP	Register as a new customer using this account number with your email address. Then you can get OTP by your email. AC: 0302345690
Role	Customer
Meter to open	MTR-00048
Read-only?	The marker account should be used to sign in and read. Please do not change another customer's profile or password. Leaderboard GET is public. Everything else under /auth/me, installations, readings, and weather needs this customer's token.

While I was building the app, a temporary debug username *customer* was left in a comment in *SessionManager*. I will confirm that account, or replace it, and put the working details in the table above before I submit. I will not ask the marker to guess.

Suggested marker check, after sign-in:

1. Home shows a site name and today's kWh.
2. History shows a chart.
3. Weather shows an hourly temperature line.
4. Earn shows an engine name of *coreml* or *json-gbdt* and a 30-day chart.
5. Account → Map shows a pin. Account → Leaderboard loads without the customer's password being sent (that call is unauthenticated).

6. Above and beyond

Multiple advanced features

Core ML and WidgetKit are both wired into Earn and Home, not bolted on as sample screens. Around them:

- **MapKit.**

  Account → Map loads every linked meter's composite and drops an annotation at the installation coordinate. The camera starts over Sri Lanka (about 7.87, 80.77) and then moves to the first site that returns coordinates. This is site data from the API, not one hardcoded pin left on the map with no model behind it.
- **EventKit.**

  "Sync high-generation days" writes all-day events for days at or above the threshold (at least 8 kWh, or 75% of the peak in the stored window, whichever is higher).
- **Charts.**

  History uses *BarMark* for daily export. Weather uses *LineMark* for temperature and *BarMark* for shortwave radiation. Earn uses *BarMark* for the 30 forecast days.
- **Anomaly detection.**

  Local comparison of the latest day with the recent positive average, with the thresholds in section 2, feeding a real notification.

Architecture and resilience

- Protocol-style forecast engines behind one *SolarForecastModel*, so the screen does not care which implementation loaded.
- Keychain for tokens and for the biometric password, with two different service names.
- Refresh-and-retry once inside the API actor.
- Offline-friendly history: the last 90 daily totals remain in SQLite. Notes remain if the network fails after they are saved. The widget still shows the last snapshot with no network of its own.
- Preferences survive relaunch. The daily summary is rebuilt on launch so a reinstall of the pending notification matches the switch.

Polish

- Shared brand lockup, card chrome, and theme tokens.
- Loading lines (“Loading your site...”, “Loading history...”, “Loading forecast...”).
- The customer’s own leaderboard row is outlined in teal.
- Password show/hide, OTP boxes, and a register sheet.
- Widget empty state, and a reload button when the widget switch is on.

Creativity

The product idea is a customer energy companion: not only “show the meter”, but “tell me if today was unusual, what tomorrow might earn, and let me pin that on the Home Screen and on my calendar.” The leaderboard and public nickname are optional and off unless the customer opts in (*nicknamePublic*).

7. Unit tests, accessibility

Unit tests

The brief requires the Swift Testing framework, not XCTest. The *HelioLankaTests* target is a unit-test bundle hosted by the app (*TEST_HOST* is *HelioLanka.app*). The five files use import Testing and *@Test*. They are included from the *HelioLankaTests* folder by the synchronized test target, and the HelioLanka scheme lists that target under Test.

The *HelioLankaTests* target contains 34 tests. The last run was green on 24 October 2026 against the iPhone 17 simulator (iOS 26.5): 34 passed, 0 failed (*xcodebuild test -scheme HelioLanka -destination 'platform=iOS Simulator,name=iPhone 17' -only-testing:HelioLankaTests*).

None of these tests call the live API. *EarningsTests* builds a *SessionManager* only so *EarnViewModel* can be created. It does not call *load()*.

AnomalyTests — 8 tests. *GenerationAnomalyEvaluator.evaluate*:

- Three days returns nil. A latest day of 0 kWh returns nil. A baseline under 2 kWh returns nil.

- A flat series (10, 10, 10, 10) returns nil. A rise to 12 against a baseline of 10 stays inside both bars and returns nil.
- 20 kWh against a baseline of 10 is *.high*, with export 20, baseline 10, and that day's start.
- 4 kWh against a baseline of 10 is *.low,* with the same baseline and day checks.
- History passed out of date order still flags the newest day (20 kWh), not whichever row happens to be last in the array.

FormatTests — 5 tests. *HelioFormat*:

- Nil energy is "- kWh". Nil power is "- kW".
- 1.5 is "1.50 kWh". 3.25 is "3.25 kW".
- 10 is "LKR 10.00". 1234.5 is "LKR 1,234.50".
- amount(-4, fractionDigits: 2) is "-4.00".

ForecastFeatureTests — **8 tests.** *SolarForecastModel.featureVector* and *dailyWeatherFeatures*:

- Empty history uses lag 6, a rolling standard deviation of 1.2, capacity as passed in, and the training weather defaults (humidity 75, temperature 28, cloud 55, rain 2 mm, shortwave 220).
- A three-day history copies lag 1 into lag 7. The rolling mean of 4, 5, and 6 is 5.
- A seven-day window uses the first value as lag 7 and the last as lag 1. The weekday index stays in 0...6. Supplied weather replaces the defaults.
- One stored day keeps the fixed spread of 1.2.
- A Monday (Calendar weekday 2) is *day_of_week* 0.
- An empty hourly list produces no daily rows. Two hours on 15 June 2026 average temperature, cloud, and radiation (20 °C and 30 °C become 25) and add precipitation (1 mm + 2 mm = 3). Humidity stays at the training default of 75 because the weather point has no humidity field.
- A point with every weather field nil uses the training defaults, except precipitation, which stays 0 when the point exists but the millimetres are missing.

EarningsTests — 9 tests. *EarnViewModel* on the main actor:

- With the history switch on, a positive stored average replaces the typed average. A nil stored average, or a stored average of 0, leaves the typed average in place.

- A negative typed average is clamped to 0. A negative typed rate is clamped to 0.
- With the site-tariff switch on, the site rate wins. With it off, the typed rate is used.
- 10 kWh/day at LKR 20 produces per day 10 kWh / LKR 200, 30 days 300 / 6,000, 90 days 900 / 18,000, and 365 days 3,650 / 73,000. The row labels are “per day”, “30 days”, “90 days”, and “365 days”.
- Forecast days of 4 kWh and 6 kWh at LKR 10 give 10 kWh and LKR 100.

SnapshotTests — 4 tests. *WidgetSnapshot*:

- A filled snapshot (meter MTR-000001, Rooftop Site 1, today 4.2 kWh, tomorrow 7.4 kWh, timestamp 1,700,000,000) round-trips through *JSONEncoder / JSONDecoder* as equal values.
- *WidgetSnapshot.empty* is meter "—", name "HelioLanka", both energies 0, and *Date.distantPast*. That empty value also round-trips.
- *WidgetSnapshot.preview* is the gallery sample: MTR-000001, Rooftop Site 1, 4.2 kWh, 7.4 kWh.

View models that only call the network are not unit-tested against the live server. That would be an integration test and would fail in a lab with no login. *UserMe* decoding and *CalendarViewModel.daysInMonth* are not in this suite.

Accessibility

After the test run I installed that Debug build on the same iPhone 17 simulator and launched it. The session was already signed in, so the first screen was Home (customer Nimal Perera, meter MTR-000001, site Rooftop Site 1), not the password form. The appearance was the forced dark scheme.

What that Home screen showed, as text a screen reader can read:

- The brand lockup. On Home the tagline is off, so the single accessibility label is “HelioLanka”. The mark image is hidden from VoiceOver.
- The title “Home” and the name “Nimal Perera”.
- The today figure as words, not a chart: “Today”, then “0.00 kWh”, then “Sum of today’s intervals”.

- Four labelled stats: Live 3.00 kW, Last interval 2.10 kWh, Cumulative 2,472.92 kWh, Capacity LKR 22.50.
- A “Recent intervals” list. Each row is a timestamp plus a kWh figure (the three rows on screen were each 2.10 kWh).
- A standard tab bar: Home, History, Weather, Earn, More.

Labels that are in the views, including screens this launch did not open:

- The password visibility control is “Hide password” or “Show password”.
- The six OTP boxes are one element: “One-time code, n of 6 digits”. VoiceOver does not land on six empty boxes.
- Where the lockup includes the tagline (sign-in and the OTP screen), the label is “HelioLanka, Solar Export Intelligence”.
- Calendar month arrows are a 44 by 44 point target. The selected day prints “Recorded export:” plus the kWh figure. The legend uses the words “High generation” and “Note”, and a caption states the high-generation threshold in kWh. The day number is text; the gold and teal dots are not the only signal.
- Widget metrics use the words Today, Tomorrow, and kWh. The empty widget says to open the app and visit Home and Earn.
- Toggles carry their titles: “Use history average kWh/day”, “Use site tariff”, “Show on leaderboard”, “Sign in with Face ID” (or Touch ID, from *BiometricAuthService.kind.settingsLabel*), “Home Screen widget”, “Remind me on this day”, “Notifications”, “High / low generation alerts”, and “Daily summary”.
- Buttons, text fields, the History segmented picker, navigation links, and the map are standard SwiftUI controls, so VoiceOver gets their titles.

8. Learning outcomes reflection

LO1 — Platform components across devices and formatss

HelioLanka is a universal target (*TARGETED_DEVICE_FAMILY* 1 and 2): iPhone and iPad.

iPhone. This is the primary device. A customer checks today's export in one hand, often in portrait. The interface is a bottom *TabView* with six tabs, a single scrolling column, 20-point screen padding, and two-column stat grids. iPhone may use portrait or either landscape. The Home Screen widget (small and medium) matters here because the answer "today / tomorrow" should be visible without opening the app. Face ID is the biometric I expect on current iPhones; the same code path selects Touch ID when that is the device biometry, which covers older iPhones and many iPads.

iPad. The same binary runs on iPad. iPad orientations include portrait, upside down, and both landscapes. The tab bar, scroll views, and *LazyVGrid* use the extra width: cards stretch, the map is larger, and charts have more horizontal room for day labels. I did not add a second navigation model such as *NavigationSplitView* or a compact-versus-regular size-class branch.

That is a deliberate scope choice, and it has a cost. A regular-width iPad in landscape still shows one phone-like column with a tab bar, so the extra canvas is used for wider cards rather than a sidebar plus detail. I accepted that because the task is a quick personal check, not a multi-document admin tool. The content that benefits from width — the map, the history chart, the forecast chart — already expands inside the scroll view. A split view would duplicate the tab structure without adding a second simultaneous task. If I extend the app, the iPad change I would make first is a regular-width layout that places the chart beside the summary, using *horizontalSizeClass == .regular,* while keeping the iPhone as a single column.

Formats beyond the two screens. The Home Screen widget is a third format: a glanceable, non-interactive summary with its own margins, no API access, and no biometric prompt. The daily notification and the calendar event are two more formats of the same daily-export fact. LO1 for this project is that one domain object (a day's export, plus tomorrow's forecast) is presented differently on the phone, on the tablet canvas, on the widget, and in Notification Center.

LO2 — Software suitable for a mobile architecture

The architecture matches the constraints of a phone:

- UI state is observable and main-actor bound. Network work is in actors.
- Secrets are in the Keychain. Preferences are in *UserDefaults*. History and notes are in SQLite. The widget sees only a snapshot in an App Group.
- The session is a single phase *enum*, so the root view cannot show tabs and the OTP screen at the same time.
- Screens do not construct *URLSession* themselves. They ask a view model, which asks a service.
- Failures become a string on the screen. The customer can pull to refresh.
- The process can be killed. Tokens, notes, daily totals, and the widget snapshot are still there on the next launch. The in-memory password exists only until the Face ID offer is answered.

I would defend this as appropriate for the size of the app. A larger coordinator or a third-party DI container would add structure I do not need for six tabs and one session.

LO3 — External APIs and devices

The customer's data is not invented on the phone. Sign-in, profile, installations, readings, weather, and the leaderboard are HTTP calls to the API in section 5. The phone handles the awkward parts of that API: OTP as a separate step, bearer tokens, one refresh retry, paged readings, and two ISO-8601 date shapes.

Device capabilities sit next to that network data rather than replacing it. Biometrics gate the saved session. Notifications remind the customer about notes and unusual days. *EventKit* can mirror a high-generation day onto the system calendar. *MapKit* plots coordinates that came from the installation composite. The widget shows numbers that Home and Earn already fetched. The forecast model consumes weather from the API and does the arithmetic locally.

LO4 — Sensors and hardware

The hardware path in this app is biometric authentication through *LocalAuthentication*, which uses the Face ID or Touch ID sensor and the Secure Enclave flow the system provides. The app does

not read raw sensor samples. It asks the system to evaluate *deviceOwnerAuthenticationWithBiometrics* and reacts to success, cancel, or "not enrolled".

Location hardware is not read by the app. Site coordinates come from the installation record, and the map displays those coordinates. I am not claiming Core Location. The calendar and notification features use system services, not motion or GPS sensors.

The on-device Core ML model is not a sensor, but it is the hardware-side computation the brief allows for advanced work: inference on the device CPU/Neural Engine, with no feature vector leaving the phone for a cloud model.

9. Declaration of LLM use

The application source code in this repository — views, view models, services, persistence, the widget, and the forecast integration — was written by me. I did not use an LLM to generate that executable code.

For this final submission I used an LLM (Grok, inside Cursor) only for non-code writing:

- Drafting this report from my existing project, so the description matches the screens and types I had already built.
- Drafting the presentation outline and speaker notes.
- Drafting the screen-recording script.
- Drafting a study guide that lists which functions I should unit test and how to add a Swift Testing target in Xcode.

I am reading and editing this report before I submit it. I can explain the architecture, the token refresh, the anomaly thresholds, the forecast feature vector, and the widget snapshot without reading from the model. The unit-test files, when they are in the project, will be written by me from that guide. I will not paste an LLM-written test file into the repository and submit it as my own implementation.

No cloud LLM is called by the running app. The solar forecast is Core ML or the bundled tree model.

10. References

Smyth, N. (2024). *IOS 18 App Development Essentials developing IOS apps with SwiftUI, Swift, and xcode 16*. Payload Media, Incorporated.

Lim, G. (2023). *Beginning iOS 16 & swift app development: Develop iOS apps, widgets with xcode 14, SwiftUI, ARKit and more*. Independently published.

M.T.Holbrook. (n.d.). *Swift AI architecture: Build intelligent iOS applications with modern design patterns*. M.T.Holbrook.

Bunda Writi. (2026). *iOS machine learning with swift*. Bunda Writi.